

Summary: Diffusion Models for MNIST Generation

GitHub repository: <https://github.com/jlvi179/Advanced-Deep-Learning>

What the Notebook Does

This notebook completes a template for a Denoising Diffusion Probabilistic Model (DDPM) to generate handwritten MNIST digits using the `denoising_diffusion_pytorch` library. It implements the core training mechanics, including calculating the diffusion loss via forward propagation, performing backpropagation, and tracking the loss over time. It also generates and visualizes intermediate digit samples to monitor the model's progress.

Architecture Choices

The main design choices revolve around the predefined U-Net configuration, the DDIM sampling, and the implementation of the training loop:

- **U-Net and DDIM:** The core model uses a custom U-Net architecture with dimension multipliers (1, 2, 5) to capture complex spatial patterns in the 28x28 images. The evaluation process is accelerated by reducing the reverse sampling timesteps from 1000 to 250 using Denoising Diffusion Implicit Models (DDIM).
- **Training loop implementation:** The core training mechanics are added to an empty template loop. This involves passing the batch of images through the `diffusion` function to calculate the loss, followed by standard PyTorch backpropagation and AdamW optimizer steps.
- **Progressive sampling:** Because a full 100-epoch run is extremely resource-intensive, intermediate images are generated regularly using the `diffusion.sample(batch_size=4)` function. This provides critical qualitative feedback on denoising capabilities long before training concludes.